

Below are **important code snippets** for each of the major deep learning methods, along with explanations of how they work and their applications. I'll use **Python** and popular deep learning frameworks like **TensorFlow** and **PyTorch**.

1. Feedforward Neural Networks (FNNs):

```
python
import tensorflow as tf
from tensorflow.keras import layers

model = tf.keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)), # Input layer
    layers.Dense(64, activation='relu'),                      # Hidden layer
    layers.Dense(10, activation='softmax')                    # Output layer
])
```

Copy

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

Explanation: A simple FNN for classification (e.g., MNIST digits). The input is flattened into a 784-dimensional vector, and the output is a probability distribution over 10 classes.

Applications: Basic classification and regression tasks.

2. Convolutional Neural Networks (CNNs):

```
python
import tensorflow as tf
from tensorflow.keras import layers

model = tf.keras.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)), # Conv layer
    layers.MaxPooling2D((2, 2)),                                           # Pooling layer
    layers.Flatten(),                                                       # Flatten
    layers.Dense(64, activation='relu'),                                    # Fully connected
    layers.Dense(10, activation='softmax')                                 # Output layer
])
```

Copy

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

Explanation: A CNN for image classification. Convolutional layers extract spatial features, and pooling layers reduce dimensionality.

Applications: Image classification, object detection.

3. Recurrent Neural Networks (RNNs):

```
python
import tensorflow as tf
from tensorflow.keras import layers

model = tf.keras.Sequential([
    layers.SimpleRNN(64, return_sequences=True, input_shape=(None, 28)), # RNN layer
    layers.SimpleRNN(64),                                                  # Another RNN layer
    layers.Dense(10, activation='softmax')                                 # Output layer
])
```

Copy

```
])
```

```
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

Explanation: An RNN for sequential data (e.g., time series or text). `return_sequences=True` ensures the output is a sequence.

Applications: Time series forecasting, text generation.

4. Transformers:

```
python
```

```
import torch
```

```
from transformers import BertTokenizer, BertForSequenceClassification
```

```
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

```
model = BertForSequenceClassification.from_pretrained('bert-base-uncased')
```

```
inputs = tokenizer("Hello, how are you?", return_tensors="pt")
```

```
outputs = model(**inputs)
```

Explanation: A transformer model (BERT) for text classification. Tokenizers convert text into input IDs, and the model outputs logits.

Applications: NLP tasks like sentiment analysis, question answering.

5. Autoencoders:

```
python
```

```
import tensorflow as tf
```

```
from tensorflow.keras import layers
```

```
encoder = tf.keras.Sequential([  
    layers.Dense(64, activation='relu', input_shape=(784,)),  
    layers.Dense(32, activation='relu')  
])
```

```
decoder = tf.keras.Sequential([  
    layers.Dense(64, activation='relu', input_shape=(32,)),  
    layers.Dense(784, activation='sigmoid')  
])
```

```
autoencoder = tf.keras.Sequential([encoder, decoder])
```

```
autoencoder.compile(optimizer='adam', loss='mse')
```

Explanation: An autoencoder compresses input data into a lower-dimensional representation (encoder) and reconstructs it (decoder).

Applications: Dimensionality reduction, anomaly detection.

6. Generative Adversarial Networks (GANs):

```
python
```

```
import tensorflow as tf
```

```
from tensorflow.keras import layers
```

Copy

Copy

Copy

Generator

```
generator = tf.keras.Sequential([
    layers.Dense(256, activation='relu', input_shape=(100,)),
    layers.Dense(784, activation='tanh')
])
```

Discriminator

```
discriminator = tf.keras.Sequential([
    layers.Dense(256, activation='relu', input_shape=(784,)),
    layers.Dense(1, activation='sigmoid')
])
```

Combined GAN

```
gan = tf.keras.Sequential([generator, discriminator])
```

Explanation: The generator creates fake data, and the discriminator tries to distinguish real from fake.

Applications: Image generation, data augmentation.

7. Variational Autoencoders (VAEs):

python

```
import tensorflow as tf
```

```
from tensorflow.keras import layers
```

Copy

Encoder

```
encoder = tf.keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(64, activation='relu')
])
```

Decoder

```
decoder = tf.keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(32,)),
    layers.Dense(784, activation='sigmoid')
])
```

VAE

```
vae = tf.keras.Sequential([encoder, decoder])
```

```
vae.compile(optimizer='adam', loss='mse')
```

Explanation: VAEs learn a probabilistic distribution of data, enabling generative capabilities.

Applications: Image generation, unsupervised learning.

8. Reinforcement Learning with Deep Neural Networks:

python

```
import torch
```

```
import torch.nn as nn
```

Copy

```
class DQN(nn.Module):
```

```
    def __init__(self, input_dim, output_dim):
```

```

super(DQN, self).__init__()
self.fc1 = nn.Linear(input_dim, 128)
self.fc2 = nn.Linear(128, 64)
self.fc3 = nn.Linear(64, output_dim)

```

```

def forward(self, x):
    x = torch.relu(self.fc1(x))
    x = torch.relu(self.fc2(x))
    return self.fc3(x)

```

Explanation: A Deep Q-Network (DQN) for reinforcement learning. It maps states to actions.

Applications: Game playing, robotics.

9. Graph Neural Networks (GNNs):

```

python
import torch
from torch_geometric.nn import GCNConv

```

Copy

```

class GCN(torch.nn.Module):
    def __init__(self):
        super(GCN, self).__init__()
        self.conv1 = GCNConv(16, 32)
        self.conv2 = GCNConv(32, 64)

    def forward(self, x, edge_index):
        x = self.conv1(x, edge_index)
        x = torch.relu(x)
        x = self.conv2(x, edge_index)
        return x

```

Explanation: A Graph Convolutional Network (GCN) for graph-structured data.

Applications: Social network analysis, molecular property prediction.

10. Self-Supervised Learning:

```

python
import torch
from torchvision import models

```

Copy

```

model = models.resnet50(pretrained=False)
# Use a pretext task (e.g., rotation prediction) to pre-train the model

```

Explanation: Self-supervised learning uses pretext tasks (e.g., predicting image rotation) to pre-train models without labeled data.

Applications: Pre-training for downstream tasks.

11. Transfer Learning:

```

python
import torch
from torchvision import models

```

Copy

```
model = models.resnet50(pretrained=True)
model.fc = torch.nn.Linear(model.fc.in_features, 10) # Replace final layer
```

Explanation: Fine-tune a pre-trained model (e.g., ResNet) for a new task.

Applications: Image classification with limited data.

12. Attention Mechanisms:

```
python
import torch
import torch.nn as nn
```

Copy

```
class Attention(nn.Module):
    def __init__(self, hidden_dim):
        super(Attention, self).__init__()
        self.attention = nn.Linear(hidden_dim, 1)

    def forward(self, x):
        attention_weights = torch.softmax(self.attention(x), dim=1)
        return (x * attention_weights).sum(dim=1)
```

Explanation: Attention mechanisms focus on important parts of the input.

Applications: NLP, computer vision.

13. Capsule Networks:

```
python
import tensorflow as tf
from tensorflow.keras import layers
```

Copy

```
class CapsuleLayer(layers.Layer):
    def __init__(self, num_capsules, dim_capsule):
        super(CapsuleLayer, self).__init__()
        self.num_capsules = num_capsules
        self.dim_capsule = dim_capsule

    def call(self, inputs):
        # Implement routing logic here
        return outputs
```

Explanation: Capsule Networks capture spatial hierarchies in data.

Applications: Image classification.

14. Neural Turing Machines (NTMs):

```
python
# Requires custom implementation
```

Copy

Explanation: NTMs combine neural networks with external memory for complex reasoning.

Applications: Algorithmic tasks.

15. Spatiotemporal Networks:

```
python
import torch
import torch.nn as nn
```

Copy

```
class ConvLSTM(nn.Module):
    def __init__(self):
        super(ConvLSTM, self).__init__()
        self.conv_lstm = nn.ConvLSTM2d(in_channels=3, out_channels=64, kernel_size=3)
```

Explanation: Combines CNNs and RNNs for spatiotemporal data.

Applications: Video analysis, activity recognition.

16. Meta-Learning:

```
python
import torch
import torch.nn as nn
```

Copy

```
class MAML(nn.Module):
    def __init__(self):
        super(MAML, self).__init__()
        self.model = nn.Sequential(nn.Linear(784, 128), nn.ReLU(), nn.Linear(128, 10))
```

Explanation: Meta-learning trains models to adapt quickly to new tasks.

Applications: Few-shot learning.

17. Few-Shot and Zero-Shot Learning:

```
python
# Requires pre-trained models like GPT or CLIP
```

Explanation: Few-shot learning uses minimal data, while zero-shot learning generalizes to unseen tasks.

Applications: NLP, image classification.

18. Neural Architecture Search (NAS):

```
python
# Requires libraries like AutoKeras or PyTorch Lightning
```

Explanation: NAS automates the design of neural network architectures.

Applications: Model optimization.

Copy

Copy